

```

Images: 2
Storage Driver:
Role: primary
Strategy: spread
Filters: health, port, dependency, affinity, constraint
Nodes: 1
docker-1: 10.240.0.2:2375
+ Status: Healthy
+ Containers: 0
+ Reserved CPUs: 0 / 1
+ Reserved Memory: 0 B / 3.794 GiB
+ Labels: executiondriver=native-0.2, kernelversion=4.2.0-18-generic, operatingsystem=Ubuntu 15.10, storedriver=devicemapper
Execution Driver:
Kernel Version:
Operating System:
CPUs: 1
Total Memory: 3.794 GiB
Name: b26a2bbb336e
ID:
Http Proxy:
Https Proxy:
No Proxy:

```

As just one node has been added to the cluster node, it shows just Node-1 added to the Swarm cluster.

Setting up Swarm Agent Node 2: [10.240.0.4]

Let's add one more agent node to see how Swarm manager actually manages multi-container nodes.

```

root@docker-2:~# docker run -d swarm join --
addr=10.240.0.4:2375 token://7733f838d176809cb2f2d24eb34ce78
cfe5b81d57f6394db655d15ce097f8ca537d6c74c2de6a477ac38544a7db
0a5d8
root@docker-2:~#

```

Finally, you can list out all the nodes added to the Swarm cluster by running the following command on the Swarm manager node as shown below:

```

#docker run -rm swarm list token://7733f838d176809cb2f2d24eb
34ce78c
10.240.0.4:2375
10.240.0.2:2375

```

So there you are...your multi-node Swarm cluster is ready to rock!

Let's create a container and run it through the Swarm manager, as follows:

```

sudo docker -H tcp://10.0.1.61:5061 run -dt --name swarm-test
nginx /bin/sh

```

```

root@docker-1:~# docker images
INFO[0999] GET /v1.18/images/json
INFO[0999] +job images()
INFO[0999] -job images() = OK (0)
REPOSITORY TAG IMAGE ID CREATED VIRTU
AL SIZE
nginx latest 813e3731b203 4 days ago 133.8
MB
centos latest 14dab3d40372 5 days ago 194.7
MB
swarm latest e0ff33e7e5b9 11 days ago 17.15
MB
ajeetraina/dell-syscfg v1.0 d121b6e6dba4 12 weeks ago 1.449
GB
tduzan/docker-omsa latest ffcdbafb4aa6 16 months ago 806.4
MB
root@docker-1:~#

```

Let's see what containers are running on each Swarm agent machine, as follows:

```

root@docker-3:~# docker -H tcp://10.240.0.5:7100 ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
NAMES
d0ee6dad4b39 nginx:latest "/bin/sh" 58 minutes ago Up 58
minutes 80/tcp,
443/tcp docker-1/swarm-test
cd9bc8c96e45 ajeetraina/dell-syscfg:v1.0 "/bin/bash" About an
hour ago Up About an hour
docker-1/jolly_curie
d0bcc8da090e bad926a6fb50:latest "/bin/bash" About an hour
ago Up About an hour
docker-2/hopeful_tesla
root@docker-3:~#

```

This means that you can easily create containers from the Swarm manager node and, interestingly, you don't need to worry about what node this container will sit on. Your Swarm cluster is intelligent enough to take care of the right resource for your container. Happy Swarming! 

References

- [1] <https://docs.docker.com/swarm/overview/>
- [2] <https://github.com/docker/swarm/releases/tag/v1.1.0>
- [3] <https://blog.docker.com/2016/02/docker-1-10>

By: Ajeet S. Raina

The author is a project lead engineer with Dell India R&D. He is a "Docker Captain" (honoured by Docker Inc.). He blogs about the latest Docker releases through his personal space www.collabnix.com. You can reach out to him through Ajeet.Raina@dell.com.